

How to convert m-file ODE to Simulink

Objectives:

- Convert an m-file model to a Simulink model

Summary of steps:

1. Copy m-file model contents to 'MATLAB Function' block
2. Specify input sized of 'MATLAB Function' block
3. Integrate $\dot{x}$, specify initial condition in integrator block

Optional steps:

4. Create a subsystem for model

Background Information:

M-files are typically used to quickly type out equations of motion and solve ODEs. When nonlinearities, such as friction, saturation and delays are considered, Simulink can be an easier tool to add these effects.

There are numerous ways to covert (port) a m-file ODE to Simulink. This tutorial shows one way to do this using the same ODE function m-file directly in Simulink.

Standard m-file ODE solution:

Solve the mass spring damper ODE $m\ddot{z} + b\dot{z} + kz = F$ with initial condition $z_0 = [0.1 \ 0]^T$ for 10 seconds.

1. Convert the ODEs to state space form $\dot{x} = f(x, u)$:

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -k/m x_1 - b/m x_2 + F/m \end{bmatrix}$$

2. Write the m-file function that evaluates $\dot{x} = f(x, u)$. This could be any form to that solves $f(x, u)$ including coefficient matrix form (even though there are special solvers for this form).

```

1 function xdot = rhs_mass_spring_damper(t,x,u)
2 % state space form for mass spring damper ODE:
3 % mxdx+bx+dx=F
4
5 m=1;k=1;b=1; % parameters
6
7 % xdot=f(x,u)
8 xdot=[x(2);
9       -k/m*x(1)-b/m*x(2)+u/m];

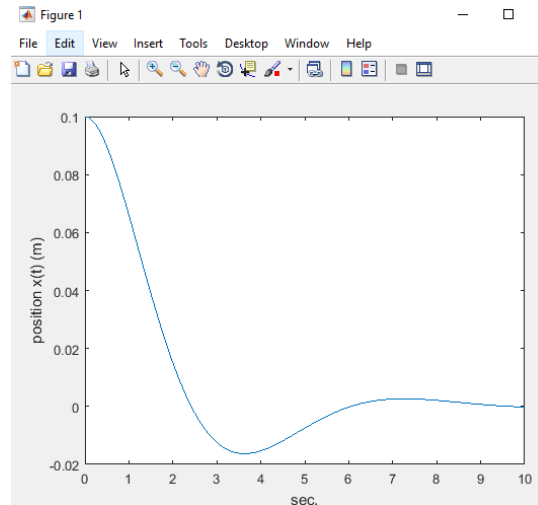
```

3. Write a m-file script that solves the ODE and plots the result:

```

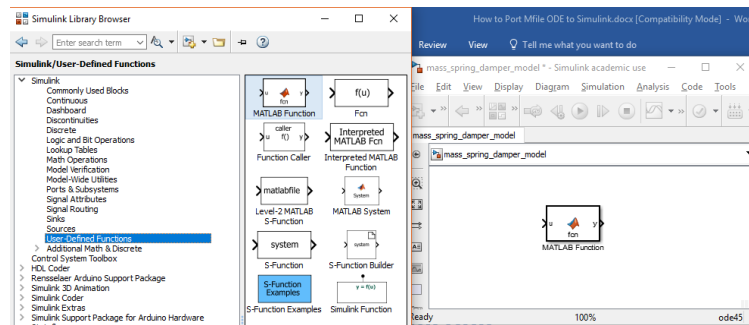
1 % solve ODE with MATLAB:
2
3 tspan=0:.01:10; % time to solve
4 x0=[.1;0];      % initial condition
5 u=0;            % input to system
6 odeoptions=[];  % ode solver options
7
8 % solve ode:
9 [t,x]=ode45(@rhs_mass_spring_damper,tspan,x0,odeoptions,u);
10
11 plot(t,x(:,1))
12 xlabel('sec. '); ylabel('position x(t) (m)')

```



Simulink using m-file ODE function:

1. Create a Simulink diagram and add 'MATLAB function' block from 'Simulink', 'User-Defined Functions':



Double click the 'MATLAB function block'

```

1 function y = fcn(u)
2 %#codegen
3
4 y = u;

```

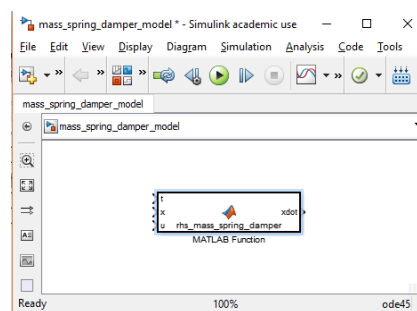
Delete all the contents. Copy and paste the first line of your m-file ODE file. Then copy this line again below but remove the “function” name in front. The actual function name is not important in this case, Simulink just used as a place holder.

```

1 function xdot = rhs_mass_spring_damper(t,x,u)
2 % The function name above is not used in Simulink
3 % is is just a placeholder
4
5 % Call the actual m-file to call the ODE
6 % Do not forget to put a semicolon at the end or it will print
7 % xdot every time step.
8 xdot = rhs_mass_spring_damper(t,x,u);

```

Notice how the MATLAB function block uses the **same exact code** as the m-file model. Save this file and the Simulink diagram will update the block. The resulting Simulink model is now:



- Depending on how xdot is defined in your original ODE m-file Simulink *may* give an error bout sizes not being defined or known. To ensure the size of xdot is known initialize the xdot in your **original** m-file to the correct size:

Original ODE m-file

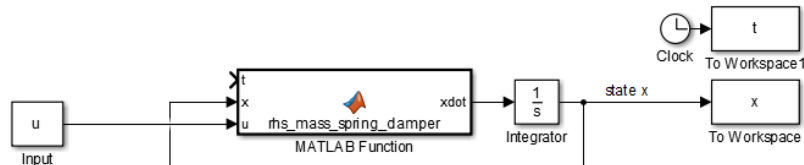
Initialize size of `xdot` to zero if Simulink error about size

```

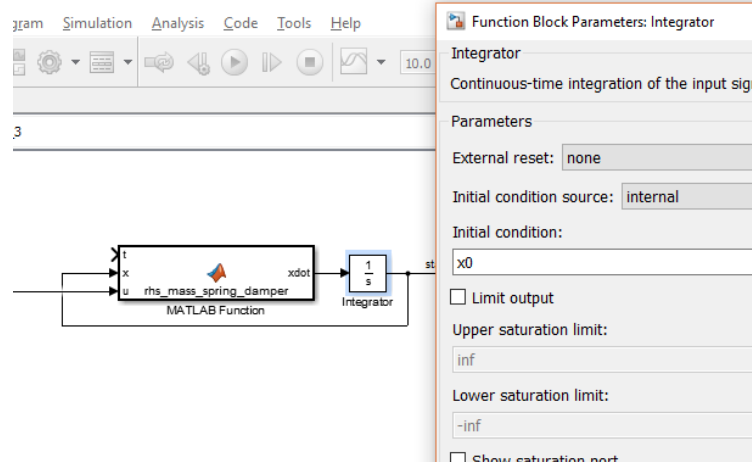
Editor - C:\Users\hurstj\OneDrive\Documents\MATLAB\How to Port Mfile ODE to Simulink R9\rhs_mass_spring_damper.m
rhs_mass_spring_damper.m
1 function xdot = rhs_mass_spring_damper(t,x,u)
2     xdot=zeros(2,1); % initialize xdot so Simulink knows the size
3
4     % state space form for mass spring damper ODE:
5     % mxdx+bxd+kx=F
6
7     m=1;k=1;b=1; % parameters
8     % xdot=f(x,u)
9     xdot=[x(2);
10          -k/m*x(1)-b/m*x(2)+u/m];

```

3. Finish 'writing' the equation $\dot{x} = f(x, u)$ by integrating $\dot{x}$, adding the input, and writing the output to the workspace (make sure the To Workspace blocks are set up as arrays)



Edit the integrator block to **specify the initial conditions:**



If you forget to specify the size of the initial conditions correctly Simulink will also give an error about size.

Simulate the Simulink diagram and plot the results:

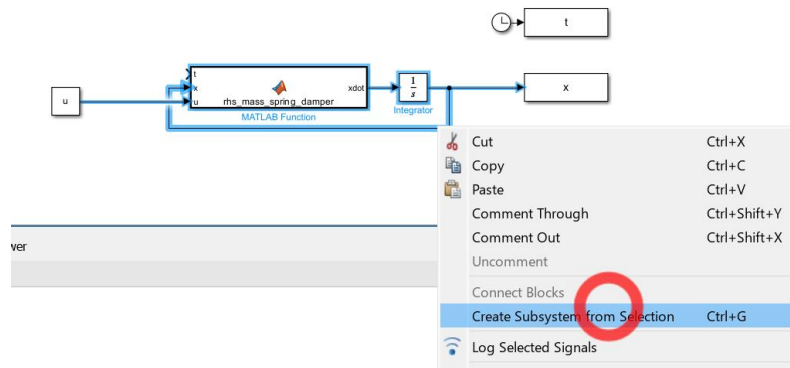
```

% Solve the ODE with simulink
sim('mass_spring_damper_model.slx')
plot(t,x(:,1))

```

Optional steps – create model subsystem

4. Select the 'MATLAB Function' and 'Integrator' block, right click, 'Create Subsystem From Selection'



Leave 't' unconnected.



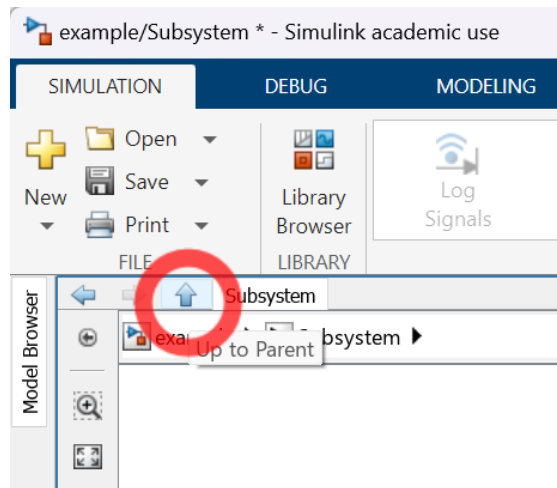
Navigate inside the subsystem and delete the port for time:



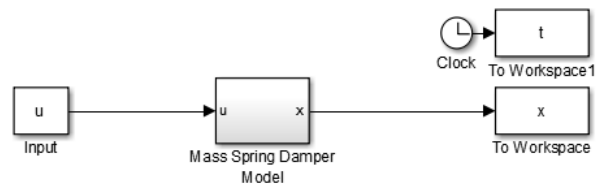
Leave 't' uconnected



Use the up arrow to navigate back to the parent block:



Now you have an easy-to-use subsystem for your block:



Optional steps – Directly specify input/output sized if you still have issues with sizing – only attempt this after carefully checking ALL the steps above

1. Carefully check all the steps above. Usually the steps below are not necessary, and can be a little harder to update if you need to change the sizes of your states or inputs.
2. You can specify the sizes of the **inputs (x and u)**. Right-click the 'MATLAB Function' block and select 'Explore':

Model Explorer

File Edit View Tools Add Help

Model Hierarchy

- Simulink Root
 - Base Workspace
 - mass_spring_damper_model*
 - Configurations
 - Model Workspace
 - External Data
 - Mass Spring Damper Model
 - MATLAB Function
 - CP_NL_plant
 - Configurations
 - Model Workspace
 - External Data
 - Subsystem
 - MATLAB Function

Contents of: ...Model/MATLAB Function (load below) Filter Contents

Column View: Stateflow Show Details 3 object(s)

Name	Scope	Port	Resolve Signal	DataType	Size
x	Input	1		Inherit: Same as Simulink	2
u	Input	2		Inherit: Same as Simulink	1
xdot	Output	1		Inherit: Same as Simulink	-1

Data u

General Description

Name: u

Scope: Input

Size: 1

Complexity: Inherited

Type: Inherit: Same as Simulink

Unit (e.g., m, m/s^2, N*m):

Inherit

Limit range

Minimum:

Leave the output size as -1. This keeps the data type of x in the Simulation diagram easy to work with.